

Reinforcement Learning for Electric Vehicles Charging

Projectwork in Multi-Agent Systems

Foschi Virginia

October 2, 2026

Abstract

This project proposes the development of a Reinforcement Learning agent capable of scheduling the overnight charging of an electric vehicle (EV) in order to minimise the electricity cost paid on the wholesale day-ahead spot market. The environment is based on real hourly wholesale electricity price data for Germany, sourced from the Ember Energy European Wholesale Electricity Price Dataset, which provides average hourly day-ahead prices for European countries over multiple years. The scenario simulates a realistic home-charging situation: an EV arrives home at a random time in the evening (between 17:00 and 20:00), and the driver needs the battery sufficiently charged by departure the following morning (07:00). During this window, an agent decides each hour whether to charge at full power, charge at half power, or do nothing. The goal is to minimise total charging cost while always ensuring the car is ready on time.

1 Introduction

Overnight EV charging does not need to start the moment a car is plugged in but it just needs to finish by the time the driver leaves the next morning. Since day-ahead electricity prices are published in advance and vary substantially from hour to hour, this flexibility can be used to reduce charging costs simply by choosing when, within that window, to draw power from the grid.

This project studies that scheduling problem and compares two approaches: a naive policy that charges immediately on arrival and a tabular Q-learning agent trained via reinforcement learning. The goal is to build the full pipeline

(data exploration, environment design, agent training, and result interpretation) and to test which design choices actually help.

Each night is treated as an independent episode. A car arrives at a random hour between 17:00 and 20:00 with a partially depleted battery, and must reach a target state of charge by 07:00 the next morning. At every hour, a charging controller must decide how much power to draw from the grid, trading off the day-ahead price at that hour against the risk of not reaching the target SoC in time. The objective is to minimize the total cost of electricity drawn over the night, without ever failing to meet the departure deadline.

2 Employed Technologies

The charging environment is implemented as a custom `gymnasium.Env` subclass, `EVChargingEnvDiscrete`, following the standard `reset()/step()` interface. This makes the environment compatible with reinforcement-learning algorithms following the Gymnasium API, even though a tabular agent is used in this project.

For data processing, `pandas` is used for loading, cleaning, and resampling the hourly electricity-price time series, as well as for performing the chronological train/validation split: prices from 2021–2023 form the training set and prices from 2024–2025 form the validation set. `numpy` is used for the physical battery model, the `Quantizer` discretization logic, and the tabular Q-table. Together, these libraries provide the numerical and data processing functionality required by the environment and learning pipeline.

`Matplotlib` is used to produce all figures in the report, including price distributions, training curves, and policy comparisons.

Finally, Jupyter notebooks are used for data exploration, analysis, and visualization. The `data_exploration.ipynb` notebook is mainly used to inspect and explore the raw electricity price data and the preprocessing steps, while `results_analysis.ipynb` is used to analyse the results obtained from the trained agent and to generate the plots presented in the report. The notebooks therefore serve primarily as analysis and experimentation interfaces, while the main environment, training, and learning components are implemented in Python.

3 The Simulation

3.1 Project Folder Structure

The project is organized into three main functional packages (data, environment and agent) together with a small number of top-level scripts:

- `data/price_loader.py` loads and cleans the raw price data, constructs the chronological train/validation split, and creates overnight episode windows

- `environment/model_env.py` implements the physical battery model, including SoC dynamics, charging efficiency, and energy cost
- `environment/discrete_env.py` implements the Gymnasium environment, the state quantizer, and the hashable discretized state representation.
- `agent/baseline.py` – contains the naive immediate-charging policy and the wrapper used to expose a trained Q-table as a policy.
- `agent/rollout_env.py` provides a shared rollout mechanism used to evaluate different policies using identical episode windows
- `agent/q_learning_agent.py` implements the tabular Q-learning algorithm, while `agent/utils.py` contains reusable schedules and experience abstractions
- `train_agent.py` contains the main training loop, hyperparameters, periodic validation, and final policy comparison
- `plot_agent.py` produces the interpretability and policy-analysis plots from a saved checkpoint
- `agent/run_baseline.py` evaluates the naive policy independently

This modular layout keeps each concern (data loading, physical simulation, policy logic, learning algorithm, and plotting) independently testable and reusable, and is what allows the same rollout utilities in `agent/rollout_env.py`, paired with the plotting functions in `plot_agent.py`, to evaluate the naive policy and the trained agent identically, without duplicated code.

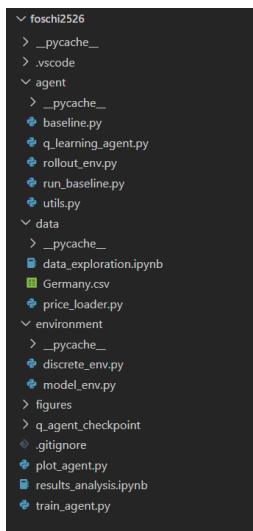


Figure 1: Project folder structure

3.2 Dataset and preprocessing

The project uses hourly German wholesale day-ahead electricity prices. The included CSV is stored as `data/Germany.csv`. The source described in the project is the Ember Energy European Wholesale Electricity Price Dataset.

The preprocessing pipeline performs the following operations:

- load the CSV file with Pandas
- convert the date and time column to a proper datetime format so that the observations can be handles as a time series
- retain the relevant datetime and price columns
- sort observations chronologically and remove duplicate timestamps
- reindex the data onto a strict hourly grid (this because the agent makes an hourly decision)
- split the series chronologically into training and validation periods
- construct valid overnight episode windows

Only the years 2021–2025 are retained from the original data. These years are then split chronologically: 2021–2023 are used for training, while 2024–2025 are kept for validation. In total, this gives 1094 valid overnight charging episodes for training and 730 episodes for validation. The agent therefore learns only from the earlier years, while the later years are kept separate to evaluate the learned policy on unseen electricity-price data.

3.3 Core Components

3.3.1 The Environment

`EVChargingEnvDiscrete` implements the charging problem as an episodic Markov Decision Process (MDP), following the standard `gymnasium` contract: `reset()` samples a new overnight episode and returns the initial discretized state; `step(action)` advances the battery model by one hour and returns the next state, the reward, and termination/truncation flags. At each hour, the agent observes the current situation, chooses a charging action, and the environment updates the battery and returns a reward. Its components are:

- **State.** The state is represented as a 3-tuple

$$(soc_bin, \text{hours_until_departure}, price_bin)$$

under the default fixed-departure configuration. SoC is discretized into 10 equal-width bins; electricity price is discretized into 3 percentile-based bins using training-data. The remaining time until departure is represented as an integer countdown.

In the original project proposal, the state also included the hour of day. However, in the final implementation this variable was removed because the departure time is fixed at 07:00. Therefore, the absolute time of day can be derived from the number of hours remaining until departure. Removing this redundant variable reduces the number of possible states and therefore keeps the Q-table smaller, while retaining the relevant temporal information for the charging decision.

- **Action.** A discrete choice among n evenly spaced charging power levels between 0 (idle) and $P_{\max} = 11$ kW, inclusive. The default configuration uses $n = 3$: idle, 5.5 kW (50%), and 11 kW (100%).

Table 1: Charging actions

Action	Meaning	Grid power
0	Idle	0 kW
1	Half charge	5.5 kW
2	Full charge	11 kW

- **Transition.** The battery capacity is 60 kWh and the charging efficiency is $\eta = 0.9$. Since every action lasts one hour, the energy taken from the grid is

$$E_t^{\text{grid}} = P(a_t)\Delta t,$$

where $P(a_t)$ is the power associated with the chosen action and $\Delta t = 1$ hour. Only 90% of this energy is stored in the battery, so

$$E_t^{\text{battery}} = \eta E_t^{\text{grid}}.$$

So the battery state of charge is then updated as

$$soc_{t+1} = \min\left(soc_t + \frac{P(a_t) \cdot \eta \cdot \Delta t}{C}, 1\right),$$

where $C = 60$ kWh is the battery capacity and $\Delta t = 1$ hour. This equation simply converts the energy added to the battery into a fraction of its total capacity. For example, charging at 11 kW for one hour gives 11 kWh from the grid. With 90% efficiency, 9.9 kWh are stored, which corresponds to an increase of $9.9/60 = 0.165$ in the SoC, i.e. about 16.5 percentage points. The $\min(\cdot, 1)$ term makes sure that the battery cannot exceed 100% SoC.

- **Reward** The reward function is designed to represent the two main goals of the charging task: minimize the cost of purchased electricity while still encouraging the vehicle to reach the required SoC before departure. The cost of charging during one hour is

$$c_t = \frac{P(a_t)p_t}{1000},$$

where p_t is the day-ahead electricity price in EUR/MWh at hour t . The division by 1000 converts the energy from kWh to MWh. The immediate reward is the negative of this cost:

$$r_t = -c_t.$$

The minus sign is important because reinforcement learning maximizes reward, whereas this problem is naturally expressed as a cost-minimization problem. A lower charging cost therefore produces a reward closer to zero, while an expensive charging action produces a more negative reward. If the electricity price is negative, the reward can instead be positive, because the cost of charging is negative. For example, charging at 11 kW for one hour when the price is 100 EUR/MWh costs $11 \times 100/1000 = 1.10$ EUR and so the immediate reward is -1.10 EUR. If the electricity price is negative, the same formula can produce a positive reward. This is consistent with the implementation, because buying electricity at a negative price corresponds to a monetary benefit rather than a cost. At the final step, a shortfall penalty is added if the vehicle has not reached the target SoC of 80%. The shortfall in energy is

$$E_{\text{shortfall}} = \max(0, 0.80 - \text{SoC}_T)C,$$

and the terminal penalty is

$$P_{\text{shortfall}} = 2E_{\text{shortfall}} \quad \text{EUR}.$$

Thus the final reward is the charging reward minus this penalty. For example, if the vehicle reaches only 70% SoC at departure, the missing amount is $0.80 - 0.70 = 0.10$ of the battery. With a 60 kWh battery, this is 6 kWh, giving a penalty of $6 \times 2 = 12$ EUR. On the other hand, if the final SoC is at least 80%, the shortfall is zero and no penalty is applied. This terminal penalty prevents the agent from reducing the electricity cost simply by remaining idle. Overall, the reward encourages the agent to find a compromise: it should charge when electricity is cheap, but it must also charge enough to reach the required SoC before departure.

- **Episode structure.** At reset step, arrival hour is sampled uniformly from $\{17, 18, 19, 20\}$; departure is fixed at 07:00 the next day by default. Arrival SoC is sampled uniformly from $[0.15, 0.45]$. Depending on the arrival time, an episode therefore last between 11 and 14 hourly steps.

Two supporting classes complete the environment:

PriceCSVLoader converts the source CSV into ready-to-use overnight episode windows. It also performs the chronological split and precomputes windows so that episode sampling does not repeatedly reconstruct the source dataframe.

EVBatteryModel is a small, environment-agnostic class holding the battery's state of charge and implementing the charging dynamics described above; it is kept separate from the **gymnasium** wrapper so it can be reused directly (e.g. for unit testing or for rolling out a fixed action sequence) without needing a full environment instance.

3.3.2 Quantization

The original SoC and electricity price values are continuous, but the project uses a tabular Q-function. **Quantizer** converts continuous values into discrete bins, so that the state can be used as a key in the Q-table. It handles the two continuous variables of the state, the SoC and the price, in different ways.

SoC bins. The SoC is a number between 0 and 1. We split this interval into 10 bins of equal width 0.1: the first bin contains the values in $[0, 0.1)$, the second the values in $[0.1, 0.2)$, and so on, up to the last bin $[0.9, 1.0]$. Since the SoC always lives in the same fixed range, equal widths are a natural choice and the bins do not depend on the data.

Price bins. The price has no fixed range, and it can even be negative, so equal-width bins would not work well. A few extreme prices would stretch the range and most observations would end up in the same bin. We therefore use percentiles. We take all the hourly prices of the training period (2021–2023) and compute their 33.3rd and 66.7th percentiles (the tertiles). These two values are the bin edges and they split the prices into three bins:

- *low*, for prices below the first edge,
- *medium*, for prices between the two edges,
- *high*, for prices above the second edge.

In this way each bin contains about one third of the training prices, so the agent sees all three price levels equally often during training. The bin boundaries are so fitted only using the training data and then reused for the validation data. This keeps the state representation consistent and avoids fitting the discretization separately to the validation set.

3.3.3 Policies and rollout

We evaluate two policies, and both use the same `act(state)` interface.

NaiveImmediateChargePolicy is the baseline. It ignores the price and charges at full power until the target SoC is reached, then stays idle.

TrainedQPolicy uses the Q-table learned by the agent. At evaluation time it always picks the action with the highest Q-value.

Since the two policies share the same interface, the function in `rollout_env.py` can run both on exactly the same chronological episodes. For every step it records the action, the SoC, the price, the reward and the cost, which are then used for the results and the plots.

3.3.4 Q-learning agent

The learning agent is implemented in `agent/q_learning_agent.py`. It uses a tabular Q-function whose entries correspond to the discrete states and the three

available actions. During training, the Q-table is updated after each environment transition using the Q-learning rule described in Section ??.

Training is performed for 30,000 episodes. The discount factor is $\gamma = 0.95$. The learning rate starts at $\alpha = 0.10$ and decreases to 0.02, while the ϵ -greedy exploration parameter decreases from 1.0 to 0.05. The Q-table is initialized with zeros and the training seed is set to 0. The final learned parameters are saved to `q_agent_checkpoint/agent_params.pkl`.

Training episodes	30,000
Discount factor γ	0.95
Initial learning rate α	0.10
Final learning rate α	0.02
Initial ϵ	1.00
Final ϵ	0.05
ϵ decay	80% of estimated total steps
Q-value initialization	0.0
Random seed	0
SoC bins	10
Price bins	3

4 Theoretical Analysis

The theoretical formulation used in this project can be described as a finite Markov Decision Process (MDP). The purpose of this section is to explain the mathematical quantities used by the implementation and how they are connected to the charging problem.

4.1 Reinforcement Learning: Problem Setting

An MDP is commonly represented as

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \quad (1)$$

where each element describes one part of the decision problem:

- $\mathcal{S}$ is the set of possible states;
- $\mathcal{A}$ is the set of possible actions;
- $P(s' | s, a)$ describes the probability of reaching state s' after taking action a in state s ;
- R is the reward function, which gives the numerical feedback received after an action;
- $\gamma \in [0, 1)$ is the discount factor, which controls the importance of future rewards.

In this project, a state contains the discretized battery state of the EV, the number of hours remaining until departure, and the discretized electricity price. The action is one of the three charging levels implemented by the environment: idle, half-power, or full-power charging.

At every time step t , the agent observes the current state s_t , selects an action a_t , receives a reward r_t , and the environment moves to a new state s_{t+1} .

The interaction can therefore be summarized as

$$s_t \xrightarrow{a_t} r_t, s_{t+1}. \quad (2)$$

The objective of the agent is not simply to maximize the reward of one individual charging action. It should choose actions that produce a good total result over the whole charging episode. For this reason, reinforcement learning uses the return

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}. \quad (3)$$

Here, G_t is the total discounted reward starting from time t . The first reward r_t is not discounted, the following reward is multiplied by γ , the next one by γ^2 , and so on. Therefore, a value of γ close to one makes future rewards important, while a smaller value gives more importance to immediate rewards. In this project, $\gamma = 0.95$, so the agent gives substantial importance to what happens later during the same charging episode.

4.2 Value Functions

A value function describes how useful a state is when the agent follows a particular policy π . The state-value function is defined as

$$V^\pi(s) = E_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right]. \quad (4)$$

In simple terms, $V^\pi(s)$ answers the question: “If the agent is currently in state s and follows policy π , how much total discounted reward can it expect from this point onward?”

For the charging problem, this means that a state is valuable when it allows the agent to obtain low-cost charging while still having enough opportunity to reach the desired battery level before departure.

The action-value function, or Q-function, adds the selected action to this definition:

$$Q^\pi(s, a) = E_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right]. \quad (5)$$

The Q-value therefore answers a more specific question: “If the agent is in state s , what is the expected future return if it first takes action a and then follows policy π ?” This is the quantity used by the project because the agent must choose between the three charging actions at every decision point.

4.3 Bellman Optimality Equation

The optimal Q-function, denoted by $Q^*(s, a)$, represents the highest expected return that can be obtained when the agent chooses actions optimally. It satisfies the Bellman optimality equation:

$$Q^*(s, a) = E \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (6)$$

The equation separates the value of an action into two parts. The first part is the immediate reward r obtained after taking the action. The second part is the best possible future value, represented by $\max_{a'} Q^*(s', a')$, discounted by γ . The expectation is used because, in a general MDP, the next state can be stochastic.

For this project, the equation provides the theoretical basis for the Q-learning update. The agent does not know the optimal Q-function beforehand. Instead, it starts with an initial Q-table and gradually improves its estimates using the rewards and next states observed during training.

The project does not explicitly learn a transition model $P(s' \mid s, a)$. Instead, the agent interacts directly with the charging environment and learns from the resulting transitions. For this reason, the implemented approach is model-free reinforcement learning.

4.4 Baseline Policy

The main reference policy is the `NaiveImmediateChargePolicy`. It does not use electricity prices to decide when to charge. When the vehicle arrives, it charges at full power until the target SoC condition is reached.

The baseline is useful because it provides a simple reference strategy. The comparison with Q-learning can then show whether learning from the price signal changes the timing of charging and affects the resulting electricity cost. The baseline itself does not estimate future prices or solve the MDP analytically.

4.5 Tabular Q-Learning

The project uses a tabular Q-learning algorithm. Instead of representing the Q-function with a neural network, the agent stores one numerical value for every state-action pair. Because the state space has been discretized and is relatively small, this representation is sufficient for the project.

After taking action a_t in state s_t , receiving reward r_t , and reaching the next state s_{t+1} , the Q-value is updated as

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \quad (7)$$

This formula can be understood step by step. The current value $Q(s_t, a_t)$ is the agent's previous estimate of how useful the selected action is. The quantity

$$r_t + \gamma \max_{a'} Q(s_{t+1}, a') \quad (8)$$

is the new target estimate. It contains the reward obtained immediately plus the best estimated future value from the next state. The difference between this target and the old Q-value,

$$\delta_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t), \quad (9)$$

is the temporal-difference error. It measures how different the new information is from what the agent previously expected.

The learning rate α determines how much of this error is used to modify the Q-value. In the project, α starts at 0.10 and is gradually reduced to 0.02. Therefore, early in training the Q-table can adapt more strongly to new experiences, while later updates are smaller.

The term $\max_{a'} Q(s_{t+1}, a')$ is particularly important. It means that the update uses the highest estimated value among all possible actions in the next state. The agent is therefore learning an estimate of the optimal action-value function rather than simply evaluating the actions selected by its exploratory behaviour. If the transition ends the episode, there is no future charging decision to evaluate. The implementation therefore removes the future-value term for terminal transitions. Equivalently, the target becomes just the final reward. This is important for the last step because the terminal shortfall penalty is part of the information that the agent must learn from.

4.6 Exploration and Exploitation

During training, the agent must balance two behaviours. It needs to exploit actions that currently have high Q-values, but it also needs to explore other actions because the initial Q-table does not contain useful knowledge.

The project uses an ϵ -greedy policy:

$$a_t = \begin{cases} \text{random action,} & \text{with probability } \epsilon, \\ \arg \max_a Q(s_t, a), & \text{with probability } 1 - \epsilon. \end{cases} \quad (10)$$

When ϵ is large, the agent explores more frequently. When ϵ is small, it mostly chooses the action with the highest Q-value. The training starts with $\epsilon = 1.0$, meaning that exploration is initially very strong. It is then reduced to 0.05 over approximately 80% of the training process.

This schedule is useful for the charging problem because the agent initially needs to try different charging times and power levels. Later, once it has learned

more about the relationship between state, price, and cost, it can rely more heavily on the learned Q-values.

At evaluation time, exploration is disabled. The trained agent therefore follows the greedy policy and selects the action with the largest Q-value for the current state. This makes the evaluation reflect the policy learned by the Q-table rather than random exploration.

4.7 State-space Design and Quantization

The Q-learning algorithm used in this project stores one value for each state-action pair. This means that the state space must be finite and sufficiently small to allow the Q-table to be represented and updated efficiently.

The original SoC and electricity price are continuous variables, so they cannot be directly used as indices of a finite Q-table. The project therefore maps them to discrete intervals. The resulting state is written as

$$s_t = (b_t^{SoC}, h_t, b_t^p), \quad (11)$$

where b_t^{SoC} is the SoC bin, h_t is the number of hours remaining until departure, and b_t^p is the electricity-price bin.

With 10 SoC bins, 3 price bins, and 14 positive countdown values, there are at most

$$10 \times 14 \times 3 = 420. \quad (12)$$

decision states. This relatively small state space is what makes a tabular approach practical here.

4.8 Reward and Learning Objective

In reinforcement learning, the reward function defines what the agent is encouraged to achieve. In this project, the reward is designed so that reducing the cost of electricity corresponds to obtaining a higher return, while failing to reach the required SoC produces an additional penalty.

The agent does not optimize each charging decision independently. Instead, it aims to maximize the cumulative discounted reward obtained over the complete charging episode. The objective can therefore be written as

$$\pi^* = \arg \max_{\pi} E \pi \left[\sum_t \gamma^t r_t \right]. \quad (13)$$

This formulation captures the main trade-off in the problem. Charging immediately can guarantee a higher SoC but may result in a higher electricity cost, while waiting for cheaper electricity can reduce the cost but leaves less time to reach the target SoC.

The terminal shortfall penalty is important because otherwise the agent could obtain a relatively high return simply by avoiding charging. By including the penalty, the learning objective reflects both aspects of the problem: reducing charging expenditure and reaching the required battery level before departure.

Therefore, the Q-learning agent learns a policy based on the long-term consequences of its decisions rather than simply selecting the cheapest action at each individual time step.

5 Results

The performance of the proposed Q-learning agent was evaluated against the naive immediate-charging policy on both the training (2021–2023) and validation (2024–2025) data. In addition to the overall cost comparison, the learning process and the resulting charging behaviour were analysed to determine whether the agent learned to exploit the temporal variation in electricity prices.

5.1 Training Performance

Figure 2 shows the evolution of the Q-learning agent during training. The blue curve represents the rolling mean reward obtained during epsilon-greedy training, while the red curve represents periodic evaluations of the frozen greedy policy on the validation data. The distinction between the two curves is important because the training reward remains affected by exploratory actions, whereas the validation evaluations measure the performance of the policy actually learned by the agent.

The greedy validation performance improves substantially throughout training. At approximately 2,000 training episodes, the validation reward is around -60 EUR per episode. It then improves progressively, reaching approximately -32 EUR at 12,000 episodes, -14 EUR at 18,000–20,000 episodes, and about -3 EUR after 22,000 episodes. The curve remains approximately stable for the remaining training, suggesting that the policy has largely converged by this point. In contrast, the noisy training reward improves more gradually from approximately -10 EUR towards approximately -5 EUR per episode.

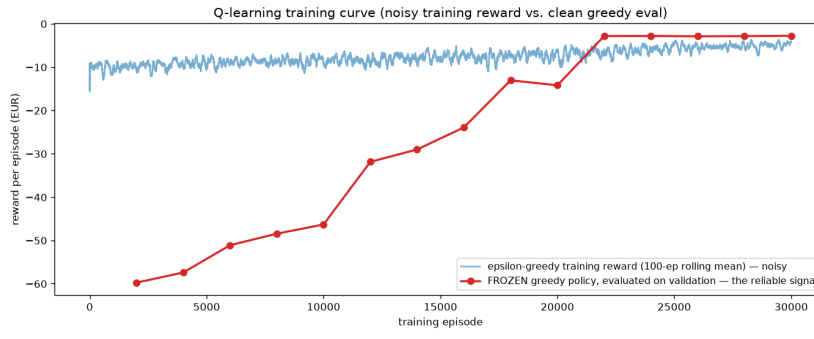


Figure 2: Q-learning training reward and periodic evaluation of the frozen greedy policy on the validation data.

5.2 Cost comparison

The final evaluation confirms the improvement quantitatively. The mean reward per night on the validation set was -2.730 EUR for Q-learning, compared with -4.380 EUR for the naive policy. Since the reward is primarily the negative electricity cost, the less negative value corresponds to lower charging cost. Thus, the learned policy reduces the average validation cost by approximately 1.65 EUR per night, corresponding to a reduction of about 38% relative to the naive policy.

The difference between training and validation performance is also visible in the final results. The mean training reward was -4.482 EUR for Q-learning, while the corresponding value for the naive policy was -6.740 EUR. The improvement is therefore present not only on the validation set but also during training. More importantly, the validation results show that the learned policy generalises to previously unseen price sequences.

Table 2: Final evaluation results for the naive and Q-learning policies.

Split	Policy	Nights	Mean cost (EUR)	Total cost (EUR)
Train	Q-learning	1094	4.482	4903.2
Train	Naive	1094	6.740	7373.3
Validation	Q-learning	730	2.730	1992.8
Validation	Naive	730	4.380	3197.3

5.3 Cumulative Charging Cost

Figure 3 compares the cumulative electricity cost of Q-learning and the naive policy over the chronologically ordered validation episodes. The two curves start from approximately the same point but diverge progressively as the number of nights increases. The naive policy accumulates approximately EUR 3200 by the end of the evaluation period, whereas the Q-learning policy accumulates approximately EUR 2000.

The increasing separation between the two curves demonstrates that the benefit of the learned policy is not caused by a small number of isolated nights. Instead, the cost advantage is accumulated consistently over the evaluation period. At the end of the validation sequence, Q-learning has therefore saved approximately EUR 1200 compared with immediate charging, corresponding to a cumulative reduction of roughly 38%.

5.4 Charging Behaviour of the Baseline

To understand the source of the cost difference, the charging behaviour of the naive policy was analysed by hour of the day. Figure 4 shows the proportion of nights in which each action was selected at each hour, together with the corresponding mean electricity price.

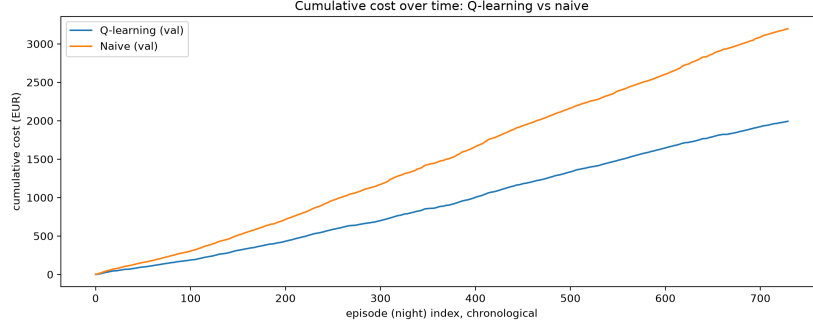


Figure 3: Cumulative charging cost of the Q-learning and naive policies over chronologically ordered validation nights

As expected from its definition, the naive policy is largely insensitive to the electricity price and charges at full power as soon as the vehicle arrives. At 18:00, 19:00 and 20:00, full-power charging is used on essentially all episodes. As the night progresses, the proportion of idle actions increases: approximately 14% of the observations are idle at 21:00, increasing to about 37% at 22:00, about 60% at 23:00 and about 90% at 00:00. This reduction in charging activity is not a price-optimisation decision; rather, it occurs because a substantial portion of vehicles have already reached their target state of charge.

The price profile illustrates why delaying charging can be beneficial. The mean price is highest around 20:00, at approximately EUR 190/MWh, while it declines during the later evening to approximately EUR 155/MWh by 23:00. Consequently, charging immediately after arrival exposes the naive policy to some of the most expensive hours in the overnight charging window.

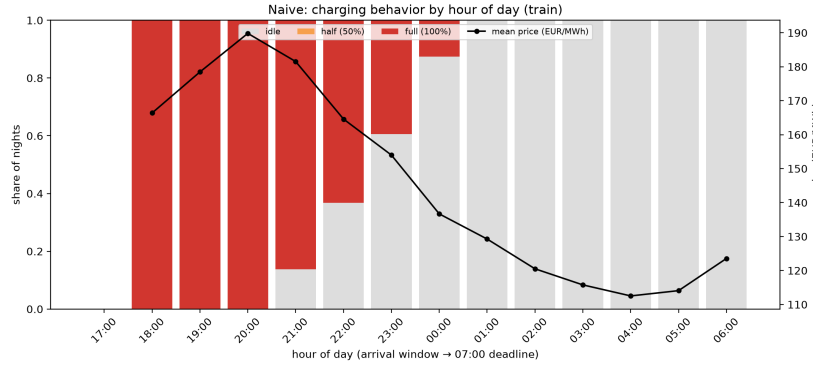


Figure 4: Charging actions selected by the naive policy by hour of day, together with the corresponding mean electricity price.

5.5 Charging Behaviour of the Learned Policy

The learned policy exhibits substantially different behaviour from the naive baseline. Figure 5 shows the same action-share breakdown for the Q-learning policy that Figure 4 showed for the naive baseline. The contrast with the naive policy is clear: instead of charging immediately after arrival, the agent tends to remain idle during the expensive early-evening hours and shifts most of its charging activity towards the later hours of the night. At 18:00 the agent is idle on about 85% of nights and charges at half power on the remaining 15%, with full power essentially never selected. At 19:00, the policy is idle on almost all nights, with only a very small share of half-power actions. From 20:00 until 00:00, the agent is almost exclusively idle. This behaviour corresponds to the relatively high electricity prices observed during the evening, which decrease substantially as the night progresses. Charging activity becomes more noticeable after midnight. At 01:00, the agent is still idle on roughly 85% of nights, while half-power charging accounts for the remaining share. At 02:00 and 03:00, both half- and full-power actions become increasingly common. By 04:00, full-power charging is the dominant action, accounting for roughly 70% of the observed actions. At 05:00 and 06:00, the agent continues to use both half and full power, while the share of idle actions remains relatively small.

The evolution of the action distribution therefore shows that the learned policy uses the available overnight flexibility to postpone charging until electricity prices are lower. Rather than charging immediately after arrival, the agent waits during the expensive evening period and progressively increases its charging activity as the departure deadline approaches. This suggests that the policy has learned to balance electricity cost against the need to reach the required state of charge before departure.

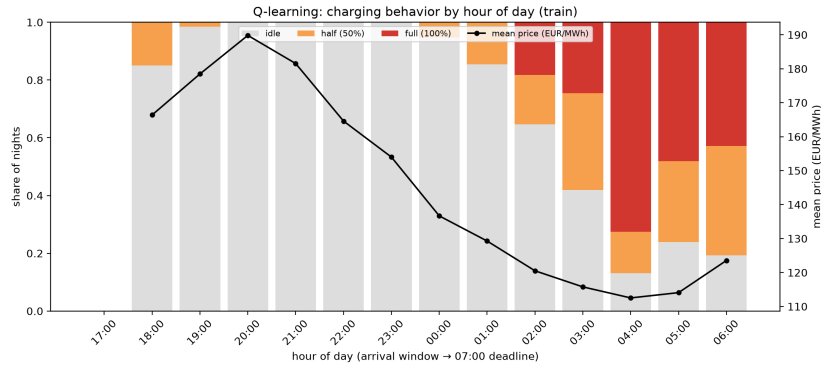


Figure 5: Charging actions selected by the Q-learning policy by hour of day, together with the corresponding mean electricity price. Only the 18:00–23:00 window is shown; the agent is idle on essentially all remaining, later hours of the night.

Figure 6 compares the average charging power selected by the Q-learning

and naive policies over the charging window. The two policies show clearly different charging patterns. The naive policy charges at close to the maximum power immediately after arrival, with an average power of about 11 kW from 18:00 to 20:00. Its charging power then decreases progressively during the night and reaches approximately zero from 01:00 onwards. In contrast, the Q-learning policy uses very little power during the early evening and increases its charging activity during the later hours.

At 18:00, the average Q-learning charging power is approximately 0.8 kW, compared with about 11 kW for the naive policy. The Q-learning power then falls to almost zero between 19:00 and 23:00, while the naive policy continues to charge, although at progressively lower power. After midnight, the behaviour of the learned policy changes: its average charging power increases and it reaches its highest average value of approximately 8.8 kW at 04:00, before decreasing slightly to around 6.8 kW at both 05:00 and 06:00.

This pattern shows that the Q-learning agent does not simply reduce the amount of charging. Instead, it changes when the charging takes place. The agent avoids most charging during the expensive early-evening period and shifts a substantial part of the required energy delivery towards the later hours, when the mean electricity price is lower. As the 07:00 departure deadline approaches, the agent increases its charging power to ensure that the vehicle can reach the required state of charge.

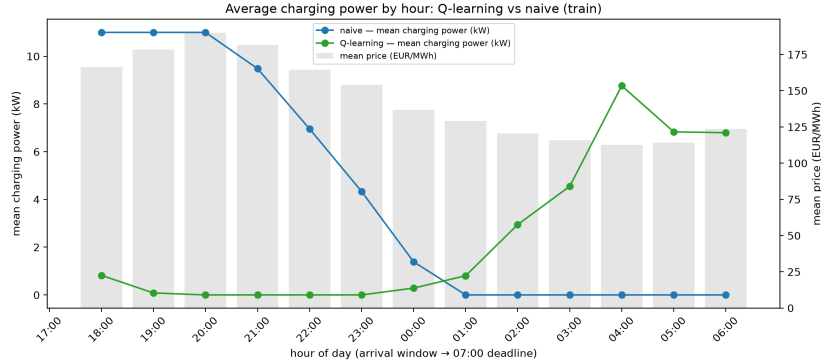


Figure 6: Average charging power selected by Q-learning and the naive policy by hour, with the corresponding mean electricity price. The Q-learning policy shifts charging towards the later hours of the night, while the naive policy charges mainly immediately after arrival.

5.6 Price Paid for Different Actions

The relationship between the selected action and the electricity price is further illustrated in Figure 7. The boxplots show the distribution of electricity prices observed when the Q-learning agent selects idle, half-power, or full-power charge-

ing. The median price for the half-power action is approximately EUR 85/MWh, while the median for the full-power action is approximately EUR 95/MWh. In comparison, the median price for idle decisions is higher, at approximately EUR 117/MWh.

The difference between the median values suggests that the agent tends to use active charging actions at lower electricity prices than those at which it remains idle.

The distributions also show that the learned policy does not use charging actions exclusively at low prices. Both active actions have relatively wide price ranges, including observations at higher prices. This is expected because the charging decision depends not only on the current electricity price but also on the remaining time until departure and the vehicle’s current state of charge.

Overall, the figure provides additional evidence that electricity price is being used by the learned policy when selecting actions. The agent tends to remain idle when prices are relatively high and is more likely to charge when prices are lower, while still adapting its decisions to the remaining charging time and required state of charge.



Figure 7: Distribution of electricity prices at which each Q-learning action is selected during training.

6 Conclusions

Overall, the experiments show that the Q-learning agent learns a charging strategy that reduces the electricity cost compared with the naive immediate-charging policy. Trained on the 2021–2023 period and evaluated on the held-out 2024–2025 period, the Q-learning agent achieves a mean validation cost of 2.730 EUR per night, compared with 4.380 EUR for the naive baseline. This corresponds to an approximately 38% reduction in the average charging cost.

The same difference is visible in the cumulative validation results, where the Q-learning policy finishes approximately EUR 1200 below the naive baseline.

The behavioural analysis helps explain this difference. The naive policy charges at close to maximum power immediately after arrival, when electricity prices are relatively high. In contrast, the Q-learning policy remains mostly idle during the early evening and shifts a larger part of its charging activity towards the later hours of the night. Its average charging power increases substantially after midnight and reaches its highest level around 04:00, before the 07:00 departure deadline. These results show that the tabular Q-learning agent learns the main scheduling trade-off in the problem: instead of charging immediately after arrival, it can wait for cheaper electricity and then increase charging power as the departure deadline approaches. The policy therefore uses both the electricity price and the remaining charging time to decide when charging should take place.

7 Reproducibility

To reproduce the results:

1. Create the required Conda environment from the supplied environment file (`environment.yml`), which specifies all required Python packages and their dependencies and activate it
2. Download the Ember Energy European Wholesale Electricity Price Dataset and extract the files
3. Place the Ember Energy Germany price CSV (`Germany.csv`) in the `data/` directory.
4. Run `python -m train_agent` to train the agent (default hyperparameters as described in Section 4.5) and reproduce the final comparison plots; the trained Q-table is saved to `q_agent_checkpoint/agent_params.pkl`. By default, training uses 2021–2023 for the training split and 2024–2025 for validation, set via the `train_years/val_years` arguments of `make_train_val_envs` in `environment/discrete.env.py`; pass different year ranges there to reproduce the results on a different split.
5. Run `result_analysis.ipynb` to analyse the trained agent and reproduce the result and interpretability plots presented in the report. The notebook loads the saved Q-table from `q_agent_checkpoint/agent_params.pkl` and evaluates the learned policy together with the naive baseline.
6. Optionally, run `python -m agent.run_baseline` to evaluate the naive baseline independently of the Q-learning agent. This provides a separate way to reproduce the baseline results without loading or training the learned Q-table.

All randomness is seeded (the agent’s own seed, and the environment’s internal random number generator, both fixed at the start of training), so re-running the training script reproduces bit-identical results